

Математика и программирование для Data Science

Третья лекция: автоматическое дифференцирование

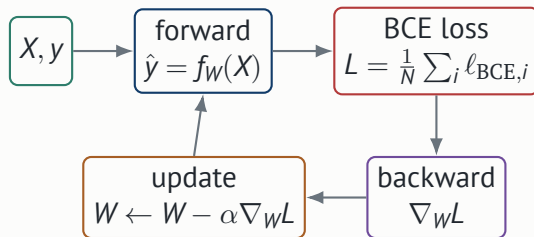
Осень 2026

Сюмбель Шангареева

На прошлых лекциях

Нейронная сеть — сложная **параметризованная функция**: её вид задаёт архитектура, а набор параметров W определяет конкретную функцию.

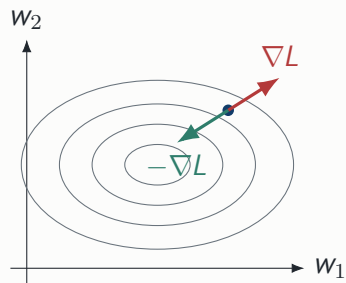
$$f_W : F \longrightarrow A, \quad \hat{y} = f_W(x).$$



Градиент

$$L = L(w_1, w_2), \quad \nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \end{bmatrix}.$$

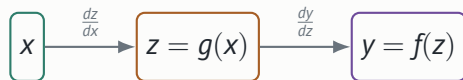
Каждая компонента —
чувствительность loss к одному
параметру.



Чтобы уменьшать $L(W)$, делаем шаг в сторону антиградиента: $W \leftarrow W - \alpha \nabla_W L$.

Производная сложной функции

$$\frac{d}{dx}f(g(x)) = f'(g(x)) g'(x)$$



$$\frac{dy}{dx} = \frac{dy}{dz} \frac{dz}{dx}$$

Программа курса

Нейронные сети и оптимизация

- Нейронная сеть
- **Производные и автоматическое дифференцирование**
- Градиентный спуск
- Стохастическая оптимизация
- Инициализация, нормализация и регуляризация

Матрички

- Спектр, чувствительность и обусловленность
- Сингулярное разложение и главные направления
- Подпространства, проекции и маломерная структура обновлений
- LoRA и квантизация
- От матриц к тензорам: Tucker, HOSVD и восстановление
- Геометрия оптимизации: предобуславливание, Shampoo и Muon

PyTorch

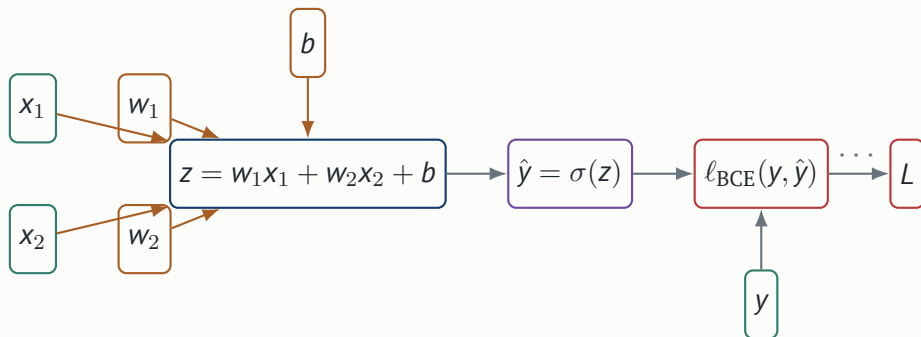
```
model = NN()
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=lr)

logits = model(X)
current_loss = criterion(logits, y)
optimizer.zero_grad()
current_loss.backward()
optimizer.step()
```

`current_loss.backward()` вычисляет $\nabla_W L$ для параметров `model`.

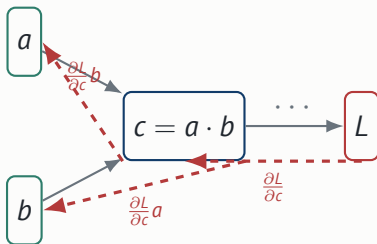
Что именно происходит внутри?

Вычислительный граф основного примера



Forward вычисляет значения в узлах. Backward вычисляет производные loss по узлам.

Локальная производная в одном узле



Локально операция умножения знает:

$$\frac{\partial c}{\partial a} = b, \quad \frac{\partial c}{\partial b} = a.$$

Если сверху уже пришло $\frac{\partial L}{\partial c}$, то chain rule даёт:

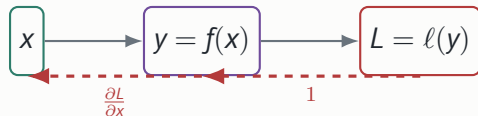
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \frac{\partial c}{\partial a}, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c} \frac{\partial c}{\partial b}.$$

Backward начинается с единицы

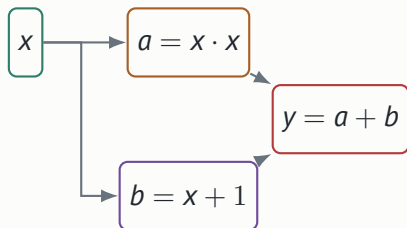
Итоговая функция L зависит от самой себя:

$$\frac{\partial L}{\partial L} = 1.$$

Это начальное значение для обратного прохода. Каждый узел получает $\frac{\partial L}{\partial \text{узел}}$ и передаёт вклад своим родителям.



Накопление градиентов



$$y = x^2 + x + 1$$
$$\frac{dy}{dx} = \underbrace{2x}_{x \rightarrow a \rightarrow y} + \underbrace{1}_{x \rightarrow b \rightarrow y}.$$

Производная по x складывает вклады всех путей от x к y .

Поэтому в коде нужен `x.grad += ...`, а не присваивание `x.grad = ....`

Что хранится в узлах вычислительного графа

1. Число

data
значение в
forward

2. Выражение

data
+, *, **
результат
операции

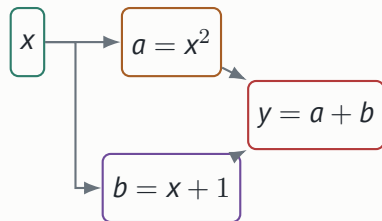
3. Связи

_prev, _op
родители и
операция

4. Производные

grad
_backward
backward()
обратный проход

Топологический обход графа



Y x два потомка: a и b . Сначала оба должны передать свои вклады в $x.\text{grad}$; только после этого вызвать $x.\text{_backward}()$.

Нужен порядок, в котором каждый узел идёт после своих родителей.

Топологический порядок вершин $v_1, \dots, v_n$ ациклического ориентированного графа — такой порядок, что для каждого ребра $u \rightarrow v$ вершина u расположена раньше вершины v .

В итоге

Когда вызываем `loss.backward()`, происходит следующее:

1. Во время forward строится граф элементарных операций.
2. Устанавливается $\frac{\partial L}{\partial L} = 1$.
3. Узлы обходятся в обратном топологическом порядке.
4. Каждый узел применяет своё локальное правило chain rule.
5. Родители накапливают все полученные вклады в градиент.

Autograd — программная реализация chain rule на вычислительном графе.

Проверка градиента

Для одного параметра w конечная разность приближает производную:

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}.$$

Сравним три значения:

1. конечные разности;
2. `Value.grad` после `backward()`;
3. `torch.tensor.grad` и конечную разность.